

Capítulo 19

Sobrecarga de Operadores; Classe string

Exercícios — Resolução Didática com Código C++

“Neste documento resolvemos os seis exercícios 19.6 a 19.11 do Capítulo 19, sendo o primeiro conceitual e os cinco restantes de programação. Os códigos exploram as três categorias principais de sobrecarga: (i) operadores aritméticos para tipos matemáticos (`Complex`, `Rational`, `Polynomial`); (ii) operadores de I/O (« e ») que se integram à infraestrutura de `cin/cout`; (iii) operadores de acesso (`operator()` para indexação bidimensional). Todos os códigos compilam com `g++ -Wall -Wextra -std=c++14` sem warnings.”

1 Exercício 19.6 — Comparando `new`, `new[]`, `delete` e `delete[]`

Enunciado. Compare os operadores de alocação e desalocação de memória dinâmica `new`, `new[]`, `delete` e `delete[]`.

Visão panorâmica

Operador	O que faz	Exemplo
<code>new T</code>	Aloca um único objeto, chama o construtor	<code>Time *t = new Time(12,0,0);</code>
<code>new T[n]</code>	Aloca array de <code>n</code> objetos, chama construtor default para cada um	<code>int *v = new int[100];</code>
<code>delete p</code>	Chama o destrutor do objeto e libera a memória	<code>delete t;</code>
<code>delete[] p</code>	Chama os destrutores de todos os elementos e libera o array inteiro	<code>delete[] v;</code>

Discussão detalhada

new T e delete p (singular)

[illegible]

A diferença fundamental do `new` em relação ao `malloc` de C é que *ele também chama o construtor*. Reciprocamente, `delete` também chama o destrutor.

`new T[n]` e `delete[] p` (array)

```
Time *vt = new Time[10];           // 10 Times default-construídos
// ... usa o array ...
delete[] vt;                       // chama ~Time() em CADA um
                                   // e libera o array inteiro
```

O erro classico: confundir as duas formas

```
int *v = new int[100];
delete v;           // <-- BUG SUTIL: comportamento INDEFINIDO
                   //                deveria ser delete[] v
```

Erro Comum de Programação

Usar `delete` no que foi alocado com `new[]`, ou `delete[]` no que foi alocado com `new`, é comportamento indefinido. O programa pode crashar, ou vazar memória, ou parecer funcionar (o pior dos cenários). A regra mnemônica: *forma do `new`, forma do `delete`*. Se uma pôs colchetes, a outra também deve pôr.

O custo do gerenciamento manual

- **Vazamento (*memory leak*):** esquecer `delete`. A memória fica reservada até o fim do programa. Programas que vazam constantemente acabam derrubando o sistema.
- **Duplo `delete`:** chamar `delete` duas vezes no mesmo ponteiro. Comportamento indefinido — frequentemente trava o programa.
- **Ponteiro pendurado (*dangling pointer*):** usar um ponteiro depois de o objeto ter sido deletado. Outra fonte clássica de crashes ou pior, *silenciosacorrupçãodedados*.

Boa Prática de Programação

Em C++ moderno (C++11+), evite `new/delete` diretos. As alternativas:

- `std::vector<T>` em vez de `new T[N] + delete[]`.
- `std::unique_ptr<T>` em vez de `new T() + delete`.
- `std::shared_ptr<T>` para propriedade compartilhada.
- `std::make_unique<T>(args...)` (C++14) ou `std::make_shared<T>(args...)` para construção segura.

Essas abstrações implementam o idioma *RAII* (*Resource Acquisition Is Initialization*): a alocação acontece no construtor, a desalocação no destrutor, e o código cliente não precisa se preocupar com `delete`. Para os exercícios deste capítulo, contudo, usamos `new/delete` crus para aprender o mecanismo de baixo nível.

2 Exercício 19.7 — Array bidimensional com operator()

Enunciado. Crie a classe `DoubleSubscriptedArray` que substitua `tabuleiroXadrez[linha][coluna]` por `tabuleiroXadrez(linha, coluna)`. Implemente: `operator()` em duas versões (uma para *lvalue* e outra *const*), `==`, `!=`, `=`, `<<` (saída em formato matricial), `>>` (entrada de todo o conteúdo). A representação interna deve ser um array linear de `linhas * colunas` elementos com aritmética de ponteiro adequada.

Análise do problema

A idéia central: armazenamos uma matriz logicamente bidimensional como um *array linear* de tamanho `linhas × colunas`. O elemento (i, j) corresponde ao índice linear $i \cdot \text{colunas} + j$ no array.

A sobrecarga de `operator()` merece duas versões:

- Uma não-const que retorna `int &` — permite `a(1,3) = 99`.
- Uma const que retorna `const int &` — permite ler de um objeto const.

Cabeçalho `DoubleSubscriptedArray.h`

```

1  // Exercício 19.7 - DoubleSubscriptedArray.h
2  // Array bidimensional com operator() para acesso (linha, coluna)
3  #ifndef DOUBLESUBSCRIPTED_H
4  #define DOUBLESUBSCRIPTED_H
5
6  #include <iostream>
7
8  class DoubleSubscriptedArray {
9      friend std::ostream &operator<<(std::ostream &out,
10                                     const DoubleSubscriptedArray &a);
11      friend std::istream &operator>>(std::istream &in,
12                                     DoubleSubscriptedArray &a);
13 public:
14     DoubleSubscriptedArray(int linhas = 3, int colunas = 3);
15     DoubleSubscriptedArray(const DoubleSubscriptedArray &outro);
16     ~DoubleSubscriptedArray();
17
18     int getLinhas() const;
19     int getColunas() const;
20
21     // operator() para acesso (linha, coluna)
22     int &operator()(int linha, int coluna);
23     const int &operator()(int linha, int coluna) const;
24
25     const DoubleSubscriptedArray &operator=(const DoubleSubscriptedArray &o);
26     bool operator==(const DoubleSubscriptedArray &o) const;
27     bool operator!=(const DoubleSubscriptedArray &o) const;
28
29 private:
30     int linhas;
31     int colunas;
32     int *dados; // array LINEAR de linhas*colunas elementos
33 };
34
35 #endif

```

Implementação `DoubleSubscriptedArray.cpp`

```

1  // Exercício 19.7 - DoubleSubscriptedArray.cpp
2  #include <iomanip>
3  #include <cstdlib>
4  #include "DoubleSubscriptedArray.h"
5  using namespace std;
6
7  DoubleSubscriptedArray::DoubleSubscriptedArray(int l, int c)
8      : linhas(l > 0 ? l : 1), colunas(c > 0 ? c : 1) {
9      dados = new int[linhas * colunas];
10     for (int i = 0; i < linhas * colunas; ++i) dados[i] = 0;
11 }
12
13 DoubleSubscriptedArray::DoubleSubscriptedArray(
14     const DoubleSubscriptedArray &o)
15     : linhas(o.linhas), colunas(o.colunas) {
16     dados = new int[linhas * colunas];
17     for (int i = 0; i < linhas * colunas; ++i) dados[i] = o.dados[i];
18 }
19
20 DoubleSubscriptedArray::~DoubleSubscriptedArray() {
21     delete[] dados;
22 }
23
24 int DoubleSubscriptedArray::getLinhas() const { return linhas; }
25 int DoubleSubscriptedArray::getColunas() const { return colunas; }
26
27 // Versao nao-const: retorna referencia mutavel (lvalue)
28 int &DoubleSubscriptedArray::operator()(int linha, int coluna) {
29     if (linha < 0 || linha >= linhas
30         || coluna < 0 || coluna >= colunas) {
31         cerr << "Indice fora dos limites." << endl;
32         exit(1);
33     }
34     return dados[linha * colunas + coluna];
35 }
36
37 // Versao const: para objetos const, retorna referencia const
38 const int &DoubleSubscriptedArray::operator()(int linha, int coluna) const {
39     if (linha < 0 || linha >= linhas
40         || coluna < 0 || coluna >= colunas) {
41         cerr << "Indice fora dos limites." << endl;
42         exit(1);
43     }
44     return dados[linha * colunas + coluna];
45 }
46
47 const DoubleSubscriptedArray &DoubleSubscriptedArray::operator=(
48     const DoubleSubscriptedArray &o) {
49     if (this != &o) { // protecao contra autoatribuicao
50         if (linhas != o.linhas || colunas != o.colunas) {
51             delete[] dados;
52             linhas = o.linhas;
53             colunas = o.colunas;
54             dados = new int[linhas * colunas];
55         }
56         for (int i = 0; i < linhas * colunas; ++i) dados[i] = o.dados[i];
57     }
58     return *this;
59 }
60
61 bool DoubleSubscriptedArray::operator==(
62     const DoubleSubscriptedArray &o) const {
63     if (linhas != o.linhas || colunas != o.colunas) return false;

```

```

64     for (int i = 0; i < linhas * colunas; ++i)
65         if (dados[i] != o.dados[i]) return false;
66     return true;
67 }
68
69 bool DoubleSubscriptedArray::operator!=(
70     const DoubleSubscriptedArray &o) const {
71     return !(*this == o);
72 }
73
74 // Operador de extracao: le todos os elementos do array
75 istream &operator>>(istream &in, DoubleSubscriptedArray &a) {
76     for (int i = 0; i < a.linhas * a.colunas; ++i) in >> a.dados[i];
77     return in;
78 }
79
80 // Operador de insercao: imprime em formato de matriz
81 ostream &operator<<(ostream &out, const DoubleSubscriptedArray &a) {
82     for (int i = 0; i < a.linhas; ++i) {
83         for (int j = 0; j < a.colunas; ++j) {
84             out << setw(6) << a.dados[i * a.colunas + j];
85         }
86         out << endl;
87     }
88     return out;
89 }

```

Programa de teste

```

1  // Exercicio 19.7 - main_DoubleSub.cpp
2  #include <iostream>
3  #include "DoubleSubscriptedArray.h"
4  using namespace std;
5
6  int main() {
7      cout << "=== Construtor (3 linhas x 5 colunas, todos zero) ===" << endl;
8      DoubleSubscriptedArray a(3, 5);
9      cout << a;
10
11     cout << "\n=== Atribuindo valores via operator(linha, coluna) ===" << endl;
12     int valor = 1;
13     for (int i = 0; i < a.getLinhas(); ++i)
14         for (int j = 0; j < a.getColunas(); ++j)
15             a(i, j) = valor++;
16     cout << a;
17
18     cout << "\n=== Acessando a(1, 3) ===" << endl;
19     cout << "a(1, 3) = " << a(1, 3) << endl;
20
21     cout << "\n=== Modificando a(2, 0) = 99 ===" << endl;
22     a(2, 0) = 99;
23     cout << a;
24
25     cout << "\n=== Teste de == e construtor de copia ===" << endl;
26     DoubleSubscriptedArray b(a); // copia
27     cout << "b (copia de a):" << endl << b;
28     cout << "a == b ? " << (a == b ? "SIM" : "NAO") << endl;
29     cout << "a != b ? " << (a != b ? "SIM" : "NAO") << endl;
30
31     b(0, 0) = 777;
32     cout << "\nApos b(0,0)=777:" << endl;
33     cout << "a == b ? " << (a == b ? "SIM" : "NAO") << endl;

```

```

34
35     cout << "\n=== Teste de atribuicao (=) com tamanhos diferentes ===" << endl;
36     DoubleSubscriptedArray c(2, 2);
37     c = a;
38     cout << "c apos c = a (c agora tem 3x5):" << endl << c;
39
40     return 0;
41 }

```

Execução

Saída da execução

```

=== Construtor (3 linhas x 5 colunas, todos zero) ===
    0    0    0    0    0
    0    0    0    0    0
    0    0    0    0    0

=== Atribuindo valores via operator(linha, coluna) ===
    1     2     3     4     5
    6     7     8     9    10
   11    12    13    14    15

=== Acessando a(1, 3) ===
a(1, 3) = 9

=== Modificando a(2, 0) = 99 ===
    1     2     3     4     5
    6     7     8     9    10
   99    12    13    14    15

=== Teste de == e construtor de copia ===
a == b ? SIM
a != b ? NAO

Apos b(0,0)=777:
a == b ? NAO

=== Teste de atribuicao (=) com tamanhos diferentes ===
c apos c = a (c agora tem 3x5):
    1     2     3     4     5
    6     7     8     9    10
   99    12    13    14    15

```

Discussão

- **Por que duas versões de `operator()`?** A versão não-`const` retorna referência ao elemento — permite modificá-lo (`a(1,3) = 9`). A versão `const` retorna `const int&` — permite leitura mas não modificação. Para uma função que recebe `const DoubleSubscriptedArray&`, somente a segunda é chamável.
- **Aritmética de ponteiro:** `dados[linha * colunas + coluna]`. É equivalente a `*(dados + linha * colunas + coluna)` — o próprio operador `[]` de C já é açúcar sintático para isso.
- **Regra dos três:** a classe gerencia memória dinâmica (`dados = new int[...]`). Por isso precisa de construtor de cópia, operador de atribuição e destrutor próprios — a chamada *Big Three* de C++03.

Observação de Engenharia de Software

A capacidade de retornar uma *referência* de uma função permite usar a chamada como *lvalue*: `a(1,3) = 99` é `a.operator()(1,3) = 99`, que é `int& = 99` — atribui 99 ao elemento aludido. Sem esse mecanismo, você teria que escrever `a.set(1,3,99)`. A sobrecarga torna a interface mais elegante.

Boa Prática de Programação

Verifique limites em operações de indexação. Nossa implementação chama `cerr` e `exit(1)` se o índice estiver fora dos limites. Em código de produção, prefere-se *lançar uma exceção* (`throw std::out_of_range(...)`) — técnica que veremos no Capítulo 22.

3 Exercício 19.8 — Classe Complex com sobrecarga total

Enunciado. Modifique a classe `Complex` para: (a) entrada/saída via `>>` e `<<` sobrecarregados (remova `print`); (b) operador de multiplicação; (c) `==` e `!=`.

Análise do problema

Trasplantamos a classe `Complex` do Cap. 17 para o paradigma de operadores sobrecarregados. As mudanças principais:

- `c1.add(c2) → c1 + c2`
- `c.print() → cout << c`
- Nova multiplicação: $(a + bi)(c + di) = (ac - bd) + (ad + bc)i$

A sobrecarga de `<<` e `>>` requer que sejam *funções amigas* (`friend`), porque o operando à esquerda é um `ostream/istream` — não um `Complex`. Como não podemos colocar essas funções como métodos de `Complex`, e elas precisam acessar membros privados, declaramo-las `friend`.

Cabeçalho `Complex.h`

```

1 // Exercício 19.8 - Complex.h
2 // Classe Complex com sobrecarga total de operadores
3 #ifndef COMPLEX_H
4 #define COMPLEX_H
5
6 #include <iostream>
7
8 class Complex {
9     // Funcoes amigas para entrada/saida
10    friend std::ostream &operator<<(std::ostream &out, const Complex &c);
11    friend std::istream &operator>>(std::istream &in, Complex &c);
12 public:
13     Complex(double real = 0.0, double imag = 0.0);
14
15     Complex operator+(const Complex &o) const;
16     Complex operator-(const Complex &o) const;
17     Complex operator*(const Complex &o) const;    // (b) multiplicacao
18
19     bool operator==(const Complex &o) const;    // (c) igualdade
20     bool operator!=(const Complex &o) const;
21
22 private:
23     double parteReal;
24     double parteImaginaria;
25 };
26
27 #endif

```

Implementação `Complex.cpp`

```

1 // Exercício 19.8 - Complex.cpp
2 #include "Complex.h"
3 using namespace std;
4
5 Complex::Complex(double real, double imag)
6     : parteReal(real), parteImaginaria(imag) {}

```



```

7
8 Complex Complex::operator+(const Complex &o) const {
9     return Complex(parteReal + o.parteReal,
10                    parteImaginaria + o.parteImaginaria);
11 }
12
13 Complex Complex::operator-(const Complex &o) const {
14     return Complex(parteReal - o.parteReal,
15                    parteImaginaria - o.parteImaginaria);
16 }
17
18 // (a+bi)*(c+di) = (ac - bd) + (ad + bc)*i
19 Complex Complex::operator*(const Complex &o) const {
20     return Complex(parteReal * o.parteReal
21                    - parteImaginaria * o.parteImaginaria,
22                    parteReal * o.parteImaginaria
23                    + parteImaginaria * o.parteReal);
24 }
25
26 bool Complex::operator==(const Complex &o) const {
27     return parteReal == o.parteReal
28            && parteImaginaria == o.parteImaginaria;
29 }
30
31 bool Complex::operator!=(const Complex &o) const {
32     return !(*this == o);
33 }
34
35 // Funcao amiga: cout << c
36 ostream &operator<<(ostream &out, const Complex &c) {
37     out << "(" << c.parteReal << ", " << c.parteImaginaria << ")";
38     return out;
39 }
40
41 // Funcao amiga: cin >> c
42 // Espera entrada no formato "real imaginaria"
43 istream &operator>>(istream &in, Complex &c) {
44     in >> c.parteReal >> c.parteImaginaria;
45     return in;
46 }

```

Programa de teste

```

1 // Exercicio 19.8 - main_Complex.cpp
2 #include <iostream>
3 #include <sstream>
4 #include "Complex.h"
5 using namespace std;
6
7 int main() {
8     Complex x; // (0, 0)
9     Complex y(4.3, 8.2);
10    Complex z(3.3, 1.1);
11
12    // Sobrecarga de << - sintaxe natural com cout
13    cout << "x: " << x << endl;
14    cout << "y: " << y << endl;
15    cout << "z: " << z << endl;
16
17    // Sobrecarga de + e cadeia de << em uma linha
18    x = y + z;
19    cout << "\nx = y + z:" << endl;

```

```

20     cout << x << " = " << y << " + " << z << endl;
21
22     // Sobrecarga de -
23     x = y - z;
24     cout << "\nx = y - z:" << endl;
25     cout << x << " = " << y << " - " << z << endl;
26
27     // (b) Sobrecarga de *
28     // (4.3 + 8.2i) * (3.3 + 1.1i) = (4.3*3.3 - 8.2*1.1) + (4.3*1.1 + 8.2*3.3)i
29     //                               = (14.19 - 9.02) + (4.73 + 27.06)i
30     //                               = (5.17, 31.79)
31     x = y * z;
32     cout << "\nx = y * z:" << endl;
33     cout << x << " = " << y << " * " << z << endl;
34
35     // (c) Comparacoes
36     cout << "\n=== Comparacoes ===" << endl;
37     Complex w(4.3, 8.2); // identico a y
38     cout << "y = " << y << ", w = " << w << endl;
39     cout << "y == w ? " << (y == w ? "SIM" : "NAO") << endl;
40     cout << "y != w ? " << (y != w ? "SIM" : "NAO") << endl;
41     cout << "y == z ? " << (y == z ? "SIM" : "NAO") << endl;
42     cout << "y != z ? " << (y != z ? "SIM" : "NAO") << endl;
43
44     // Sobrecarga de >> usando stringstream para simular entrada
45     cout << "\n=== Teste de operator>> (extracao de stream) ===" << endl;
46     Complex c;
47     stringstream entrada("2.5 -1.7");
48     entrada >> c;
49     cout << "Apos >> lendo '2.5 -1.7': c = " << c << endl;
50
51     return 0;
52 }

```

Execução

Saída da execução

```

x: (0, 0)
y: (4.3, 8.2)
z: (3.3, 1.1)

x = y + z:
(7.6, 9.3) = (4.3, 8.2) + (3.3, 1.1)

x = y - z:
(1, 7.1) = (4.3, 8.2) - (3.3, 1.1)

x = y * z:
(5.17, 31.79) = (4.3, 8.2) * (3.3, 1.1)

=== Comparacoes ===
y == w ? SIM
y != w ? NAO
y == z ? NAO
y != z ? SIM

=== Teste de operator>> (extracao de stream) ===
Apos >> lendo '2.5 -1.7': c = (2.5, -1.7)

```

Conferência matemática da multiplicação

$$(4,3+8,2i)(3,3+1,1i) = (4,3 \cdot 3,3 - 8,2 \cdot 1,1) + (4,3 \cdot 1,1 + 8,2 \cdot 3,3)i = (14,19 - 9,02) + (4,73 + 27,06)i = 5,17 + 31,79i.$$

Discussão — amigas para I/O

A *convenção firme* em C++ é:

```
ostream& operator<<(ostream &out, const T &valor); // sempre nesta forma
istream& operator>>(istream &in, T &valor);      // sempre nesta forma
```

Por que retornar `ostream&` e não `void`? Para permitir o encadeamento:

```
cout << x << " = " << y << " + " << z << endl;
//      ~~~~~^ retornou cout
//      ~~~~~^ retornou cout
//      ~~~~~^ retornou cout
//      [...]
```

Cada chamada de « retorna `cout` para que a próxima possa ser aplicada. Sem isso, teríamos que escrever cada inserção numa linha separada.

Observação de Engenharia de Software

A sobrecarga de « e » é o *exemplo paradigmático* do uso legítimo de `friend`. Tem que ser função livre (porque o operando esquerdo é `ostream`, não o tipo definido pelo usuário), mas precisa acessar os membros privados para formatar. Sem `friend`, teríamos que oferecer *getters* para cada membro — expondo a representação interna ao código cliente.

4 Exercício 19.9 — Classe HugeInt expandida

Enunciado. Considere a classe `HugeInt` das figuras 19.23–19.25. (a) Descreva como ela opera. (b) Que restrições ela tem? (c) Sobrecarregue `*`. (d) Sobrecarregue `/`. (e) Sobrecarregue todos os operadores relacionais.

(a) Como a classe `HugeInt` opera

A classe armazena um inteiro de até 30 dígitos como um array de `short[30]`, onde `integer[i]` contém o *i*-ésimo dígito — mais especificamente, `integer[digits-1]` é a unidade, `integer[digits-2]` é a dezena, e assim por diante. É uma representação “big-endian” em decimal.

A soma percorre o array da direita para a esquerda (do menos significativo para o mais), efetuando a aritmética escolar com *carry* de 1 sempre que o resultado parcial exceder 9.

Há três sobrecargas do `operator+`: `HugeInt + HugeInt`, `HugeInt + int`, `HugeInt + string`. As duas últimas funcionam convertendo o operando direito em `HugeInt` e chamando a primeira recursivamente.

A saída via `operator«` é função amiga: percorre o array da esquerda para a direita, pulando zeros iniciais.

(b) Restrições

- **Capacidade máxima:** 30 dígitos. Resultados que ultrapassem (e.g., $999\dots9 + 1$, com 30 noves) sofrem *overflow silencioso* — o *carry* final é descartado.
- **Apenas números não-negativos:** a classe original não representa sinais; não há operação de subtração na figura 19.24.
- **Sem operações de comparação** (na versão original do livro). Vamos adicioná-las no item (e).
- **Sem multiplicação nem divisão** (idem). Vamos adicioná-las nos itens (c) e (d).
- **Custo de memória fixo:** cada `HugeInt` ocupa 30 `shorts` (60 bytes em arquiteturas típicas), independente do número de dígitos efetivamente usados.

(c)-(e) Cabeçalho `HugeInt.h` expandido

```

1  // Exercício 19.9 - HugeInt.h
2  // Inteiros gigantes (30 dígitos) com: + (livro), * /, ==, !=, <, >, <=, >=
3  #ifndef HUGEINT_H
4  #define HUGEINT_H
5
6  #include <iostream>
7  #include <string>
8
9  class HugeInt {
10     friend std::ostream &operator<<(std::ostream &, const HugeInt &);
11 public:
12     static const int digits = 30;
13
14     HugeInt(long valor = 0);           // converte de long
15     HugeInt(const std::string &numero); // converte de string
16
17     HugeInt operator+(const HugeInt &op2) const;
18     HugeInt operator+(int op2) const;
19     HugeInt operator+(const std::string &op2) const;
20

```

```

21 // (c) Multiplicacao e (d) divisao
22 HugeInt operator*(const HugeInt &op2) const;
23 HugeInt operator/(const HugeInt &op2) const;
24
25 // Subtracao (necessaria para a divisao)
26 HugeInt operator-(const HugeInt &op2) const;
27
28 // (e) Relacionais e igualdade
29 bool operator==(const HugeInt &o) const;
30 bool operator!=(const HugeInt &o) const;
31 bool operator< (const HugeInt &o) const;
32 bool operator> (const HugeInt &o) const;
33 bool operator<=(const HugeInt &o) const;
34 bool operator>=(const HugeInt &o) const;
35
36 bool isZero() const;
37
38 private:
39     short integer[digits]; // integer[digits-1] = unidade
40 };
41
42 #endif

```

Implementação expandida

```

1 // Exercício 19.9 - HugeInt.cpp
2 #include <cctype>
3 #include "HugeInt.h"
4 using namespace std;
5
6 HugeInt::HugeInt(long value) {
7     for (int i = 0; i < digits; ++i) integer[i] = 0;
8     for (int j = digits - 1; value != 0 && j >= 0; --j) {
9         integer[j] = value % 10;
10        value /= 10;
11    }
12 }
13
14 HugeInt::HugeInt(const string &number) {
15     for (int i = 0; i < digits; ++i) integer[i] = 0;
16     int length = static_cast<int>(number.size());
17     for (int j = digits - length, k = 0; j < digits; ++j, ++k) {
18         if (isdigit(static_cast<unsigned char>(number[k])))
19             integer[j] = number[k] - '0';
20     }
21 }
22
23 HugeInt HugeInt::operator+(const HugeInt &op2) const {
24     HugeInt temp;
25     int carry = 0;
26     for (int i = digits - 1; i >= 0; --i) {
27         temp.integer[i] = integer[i] + op2.integer[i] + carry;
28         if (temp.integer[i] > 9) { temp.integer[i] %= 10; carry = 1; }
29         else
30             carry = 0;
31     }
32     return temp;
33 }
34
35 HugeInt HugeInt::operator+(int op2) const {
36     return *this + HugeInt(op2);
37 }

```

```

38 HugeInt HugeInt::operator+(const string &op2) const {
39     return *this + HugeInt(op2);
40 }
41
42 // Subtracao: assume *this >= op2 (resultado nao-negativo)
43 HugeInt HugeInt::operator-(const HugeInt &op2) const {
44     HugeInt temp;
45     int borrow = 0;
46     for (int i = digits - 1; i >= 0; --i) {
47         int d = integer[i] - op2.integer[i] - borrow;
48         if (d < 0) { d += 10; borrow = 1; }
49         else borrow = 0;
50         temp.integer[i] = d;
51     }
52     return temp;
53 }
54
55 // (c) Multiplicacao: O(digits^2) - estilo escolar
56 HugeInt HugeInt::operator*(const HugeInt &op2) const {
57     HugeInt temp;
58     for (int i = digits - 1; i >= 0; --i) {           // digito de *this
59         if (integer[i] == 0) continue;
60         int carry = 0;
61         for (int j = digits - 1; j >= 0; --j) {       // digito de op2
62             int posicao = i + j - (digits - 1);      // posicao no resultado
63             if (posicao < 0) {
64                 // ultrapassa o tamanho - ignora (overflow)
65                 if (integer[i] * op2.integer[j] + carry != 0) {
66                     // overflow silencioso
67                 }
68                 continue;
69             }
70             int s = temp.integer[posicao]
71                 + integer[i] * op2.integer[j] + carry;
72             temp.integer[posicao] = s % 10;
73             carry = s / 10;
74         }
75     }
76     return temp;
77 }
78
79 // (d) Divisao: por subtracoes repetidas (simples mas ineficiente)
80 HugeInt HugeInt::operator/(const HugeInt &op2) const {
81     HugeInt quociente;
82     if (op2.isZero()) return quociente;              // divisao por zero -> 0
83     HugeInt resto = *this;
84     while (resto >= op2) {
85         resto = resto - op2;
86         quociente = quociente + HugeInt(1L);
87     }
88     return quociente;
89 }
90
91 bool HugeInt::isZero() const {
92     for (int i = 0; i < digits; ++i)
93         if (integer[i] != 0) return false;
94     return true;
95 }
96
97 // (e) Relacionais
98 bool HugeInt::operator==(const HugeInt &o) const {
99     for (int i = 0; i < digits; ++i)
100         if (integer[i] != o.integer[i]) return false;
101     return true;

```

```

102 }
103
104 bool HugeInt::operator!=(const HugeInt &o) const { return !(*this == o); }
105
106 bool HugeInt::operator>(const HugeInt &o) const {
107     for (int i = 0; i < digits; ++i) {
108         if (integer[i] > o.integer[i]) return true;
109         if (integer[i] < o.integer[i]) return false;
110     }
111     return false;
112 }
113
114 bool HugeInt::operator<(const HugeInt &o) const { return o > *this; }
115 bool HugeInt::operator>=(const HugeInt &o) const { return !(*this < o); }
116 bool HugeInt::operator<=(const HugeInt &o) const { return !(*this > o); }
117
118 ostream &operator<<(ostream &output, const HugeInt &num) {
119     int i;
120     for (i = 0; num.integer[i] == 0 && i <= HugeInt::digits; ++i)
121         ; // pula zeros a esquerda
122     if (i == HugeInt::digits) output << 0;
123     else for (; i < HugeInt::digits; ++i) output << num.integer[i];
124     return output;
125 }

```

Programa de teste

```

1 // Exercício 19.9 - main_HugeInt.cpp
2 #include <iostream>
3 #include "HugeInt.h"
4 using namespace std;
5
6 int main() {
7     cout << "=== Soma (livro) ===" << endl;
8     HugeInt n1(7654321);
9     HugeInt n2(7891234);
10    HugeInt n3("99999999999999999999999999999999");
11    HugeInt n4("1");
12    HugeInt n5;
13    cout << "n1 = " << n1 << endl;
14    cout << "n2 = " << n2 << endl;
15    cout << "n3 = " << n3 << endl;
16    cout << n1 << " + " << n2 << " = " << n1 + n2 << endl;
17    cout << n3 << " + " << n4 << "\n = " << (n3 + n4) << endl;
18
19    // (c) Multiplicacao
20    cout << "\n=== Multiplicacao ===" << endl;
21    HugeInt a(123);
22    HugeInt b(456);
23    cout << a << " * " << b << " = " << (a * b) << "      (esperado 56088)" << endl;
24
25    HugeInt grande("1000000000000");           // 10^12
26    HugeInt outro("1000000000000");
27    cout << grande << " * " << outro << "\n = " << (grande * outro)
28        << "      (esperado 10^24)" << endl;
29
30    // (d) Divisao
31    cout << "\n=== Divisao ===" << endl;
32    HugeInt d1(1000);
33    HugeInt d2(7);
34    cout << d1 << " / " << d2 << " = " << (d1 / d2)
35        << "      (esperado 142)" << endl;

```

```

36 HugeInt d3("1000000");
37 HugeInt d4(1000);
38
39 cout << d3 << " / " << d4 << " = " << (d3 / d4)
40     << " (esperado 1000)" << endl;
41
42 // (e) Relacionais
43 cout << "\n=== Operadores relacionais ===" << endl;
44 HugeInt x(100);
45 HugeInt y(200);
46 HugeInt z(100);
47 cout << "x=" << x << " y=" << y << " z=" << z << endl;
48 cout << "x == z ? " << (x == z ? "SIM" : "NAO") << endl;
49 cout << "x != y ? " << (x != y ? "SIM" : "NAO") << endl;
50 cout << "x < y ? " << (x < y ? "SIM" : "NAO") << endl;
51 cout << "x > y ? " << (x > y ? "SIM" : "NAO") << endl;
52 cout << "x <= z ? " << (x <= z ? "SIM" : "NAO") << endl;
53 cout << "y >= x ? " << (y >= x ? "SIM" : "NAO") << endl;
54
55 return 0;
56 }

```

Execução

Saída da execução

[illegible]

Conferência matemática

- $123 \times 456 = 56088$.
- $10^{12} \times 10^{12} = 10^{24}$. Confere com a saída (24 zeros após o 1).
- $1000 \div 7 = 142$ (parte inteira, pois $7 \times 142 = 994$ e $7 \times 143 = 1001$).
- $10000000 \div 1000 = 10000$.

Discussão didática das implementações

- **Multiplicação:** algoritmo escolar $O(n^2)$. Para cada par de dígitos (i, j) , computa-se o produto e adiciona-se na posição apropriada do resultado, com propagação de *carry*. Para inteiros maiores, há algoritmos mais eficientes (Karatsuba $O(n^{1.585})$, Toom-Cook, FFT). Para 30 dígitos, o algoritmo escolar é adequado.
- **Divisão:** implementamos por *subtrações repetidas* — conta-se quantas vezes o divisor cabe no dividendo. É conceitualmente claro mas *terrivelmente ineficiente* para divisões onde o quociente é grande. Por exemplo, $10^{20}/1$ exigiria 10^{20} subtrações. Em produção, usar-se-ia a divisão escolar (*long division*) ou métodos com aproximações sucessivas.
- **Subtração:** foi necessária para a divisão. Assumimos $a \geq b$ (resultado não-negativo); caso contrário, comportamento indefinido.

Dica de Desempenho

Para `HugeInts` pequenos como os do programa de teste, o algoritmo simples é rápido o suficiente. Para inteiros realmente grandes (milhares ou milhões de dígitos, como os usados em criptografia RSA), usa-se bibliotecas como `GMP` ou `Boost.Multiprecision` que implementam algoritmos sofisticados em *assembly* otimizado.

5 Exercício 19.10 — Classe RationalNumber

Enunciado. Crie a classe `RationalNumber` (frações) com: (a) construtor que impede denominador 0, reduz frações e evita denominadores negativos; (b) sobrecarga de `+`, `-`, `*`, `/`; (c) sobrecarga dos operadores relacionais e de igualdade.

Análise do problema

Esta é a versão sobrecarregada da classe `Rational` do Cap. 17. As novidades:

- `r1.add(r2) → r1 + r2`
- `r1.printFraction() → cout << r1`
- Adição de `operator>>` para entrada
- Operadores relacionais (`<`, `>`, `<=`, `>=`) que comparam corretamente sem perda de precisão

Cabeçalho `RationalNumber.h`

```

1 // Exercício 19.10 - RationalNumber.h
2 // Fracao com sobrecarga total de operadores: + - * / == != < > <= >=
3 // e << >> para I/O
4 #ifndef RATIONALNUMBER_H
5 #define RATIONALNUMBER_H
6
7 #include <iostream>
8
9 class RationalNumber {
10     friend std::ostream &operator<<(std::ostream &out, const RationalNumber &r);
11     friend std::istream &operator>>(std::istream &in, RationalNumber &r);
12 public:
13     RationalNumber(int numerador = 0, int denominador = 1);
14
15     // Aritmeticos
16     RationalNumber operator+(const RationalNumber &o) const;
17     RationalNumber operator-(const RationalNumber &o) const;
18     RationalNumber operator*(const RationalNumber &o) const;
19     RationalNumber operator/(const RationalNumber &o) const;
20
21     // Relacionais e igualdade
22     bool operator==(const RationalNumber &o) const;
23     bool operator!=(const RationalNumber &o) const;
24     bool operator< (const RationalNumber &o) const;
25     bool operator> (const RationalNumber &o) const;
26     bool operator<=(const RationalNumber &o) const;
27     bool operator>=(const RationalNumber &o) const;
28
29 private:
30     int num;    // numerador (carrega o sinal)
31     int den;    // denominador (sempre > 0)
32
33     void reduzir();
34     static int mdc(int a, int b);
35 };
36
37 #endif

```

Implementação RationalNumber.cpp

```

1  // Exercício 19.10 - RationalNumber.cpp
2  #include <cstdlib>
3  #include "RationalNumber.h"
4  using namespace std;
5
6  RationalNumber::RationalNumber(int numerador, int denominador)
7      : num(numerador), den(denominador) {
8      if (den == 0) {
9          cerr << "Erro: denominador 0. Ajustado para 1." << endl;
10         den = 1;
11     }
12     if (den < 0) { num = -num; den = -den; } // sinal sempre no numerador
13     reduzir();
14 }
15
16 int RationalNumber::mdc(int a, int b) {
17     a = abs(a); b = abs(b);
18     while (b != 0) { int t = b; b = a % b; a = t; }
19     return a == 0 ? 1 : a;
20 }
21
22 void RationalNumber::reduzir() {
23     int d = mdc(num, den);
24     num /= d;
25     den /= d;
26 }
27
28 RationalNumber RationalNumber::operator+(const RationalNumber &o) const {
29     return RationalNumber(num * o.den + o.num * den, den * o.den);
30 }
31
32 RationalNumber RationalNumber::operator-(const RationalNumber &o) const {
33     return RationalNumber(num * o.den - o.num * den, den * o.den);
34 }
35
36 RationalNumber RationalNumber::operator*(const RationalNumber &o) const {
37     return RationalNumber(num * o.num, den * o.den);
38 }
39
40 RationalNumber RationalNumber::operator/(const RationalNumber &o) const {
41     return RationalNumber(num * o.den, den * o.num);
42 }
43
44 // Para comparar a/b ?= c/d, evitamos divisao em ponto flutuante:
45 // compara a*d com c*b (lembrar que ambos den > 0)
46 bool RationalNumber::operator==(const RationalNumber &o) const {
47     return num == o.num && den == o.den; // ambos ja reduzidos!
48 }
49 bool RationalNumber::operator!=(const RationalNumber &o) const {
50     return !(*this == o);
51 }
52 bool RationalNumber::operator<(const RationalNumber &o) const {
53     return num * o.den < o.num * den;
54 }
55 bool RationalNumber::operator>(const RationalNumber &o) const { return o < *this; }
56 bool RationalNumber::operator<=(const RationalNumber &o) const { return !(*this > o); }
57 bool RationalNumber::operator>=(const RationalNumber &o) const { return !(*this < o); }
58
59 // I/O

```

```

60 ostream &operator<<(ostream &out, const RationalNumber &r) {
61     if (r.den == 1) out << r.num;
62     else out << r.num << "/" << r.den;
63     return out;
64 }
65
66 // Le no formato "num / den" ou "num"
67 istream &operator>>(istream &in, RationalNumber &r) {
68     int n, d = 1;
69     in >> n;
70     if (in.peek() == '/') {
71         in.get(); // descarta '/'
72         in >> d;
73     }
74     r = RationalNumber(n, d);
75     return in;
76 }

```

Programa de teste

```

1 // Exercício 19.10 - main_RationalNumber.cpp
2 #include <iostream>
3 #include "RationalNumber.h"
4 using namespace std;
5
6 int main() {
7     cout << "=== Construtores: 2/4 e 3/9 sao reduzidos ===" << endl;
8     RationalNumber r1(2, 4);
9     RationalNumber r2(3, 9);
10    cout << "2/4 -> " << r1 << endl; // 1/2
11    cout << "3/9 -> " << r2 << endl; // 1/3
12
13    cout << "\n=== Aritmetica ===" << endl;
14    cout << r1 << " + " << r2 << " = " << (r1 + r2) << endl;
15    cout << r1 << " - " << r2 << " = " << (r1 - r2) << endl;
16    cout << r1 << " * " << r2 << " = " << (r1 * r2) << endl;
17    cout << r1 << " / " << r2 << " = " << (r1 / r2) << endl;
18
19    cout << "\n=== Comparacoes ===" << endl;
20    RationalNumber a(1, 2);
21    RationalNumber b(2, 4); // mesma fracao reduzida que a
22    RationalNumber c(1, 3);
23    cout << "a = " << a << ", b = " << b << ", c = " << c << endl;
24    cout << "a == b ? " << (a == b ? "SIM" : "NAO") << endl;
25    cout << "a != c ? " << (a != c ? "SIM" : "NAO") << endl;
26    cout << "a > c ? " << (a > c ? "SIM" : "NAO") << " (1/2 > 1/3)" << endl;
27    cout << "a < c ? " << (a < c ? "SIM" : "NAO") << endl;
28    cout << "a >= b ? " << (a >= b ? "SIM" : "NAO") << endl;
29    cout << "c <= a ? " << (c <= a ? "SIM" : "NAO") << endl;
30
31    cout << "\n=== Validacao: denominador 0 ===" << endl;
32    RationalNumber bug(5, 0);
33    cout << "5/0 -> " << bug << endl;
34
35    cout << "\n=== Sinal sempre no numerador ===" << endl;
36    RationalNumber neg(2, -5);
37    cout << "2/-5 -> " << neg << endl;
38
39    return 0;
40 }

```

Execução

Saída da execução

```

=== Construtores: 2/4 e 3/9 sao reduzidos ===
2/4 -> 1/2
3/9 -> 1/3

=== Aritmetica ===
1/2 + 1/3 = 5/6
1/2 - 1/3 = 1/6
1/2 * 1/3 = 1/6
1/2 / 1/3 = 3/2

=== Comparacoes ===
a = 1/2, b = 1/2, c = 1/3
a == b ? SIM
a != c ? SIM
a > c ? SIM (1/2 > 1/3)
a < c ? NAO
a >= b ? SIM
c <= a ? SIM

=== Validacao: denominador 0 ===
Erro: denominador 0. Ajustado para 1.
5/0 -> 5

=== Sinal sempre no numerador ===
2/-5 -> -2/5

```

Conferência matemática

$$\frac{1}{2} + \frac{1}{3} = \frac{3+2}{6} = \frac{5}{6}; \quad \frac{1}{2} \cdot \frac{1}{3} = \frac{1}{6}; \quad \frac{1}{2} \div \frac{1}{3} = \frac{3}{2}.$$

Discussão — comparação de frações sem perda de precisão

A comparação $a/b < c/d$ poderia ser feita como `static_cast<double>(num) / den < static_cast<double>(o.num) / o.den`, mas isso introduz erros de ponto flutuante. A maneira matematicamente exata: como $b > 0$ e $d > 0$ (denominadores sempre positivos), multiplicar ambos os lados por $b \cdot d > 0$ preserva a desigualdade:

$$\frac{a}{b} < \frac{c}{d} \iff a \cdot d < c \cdot b.$$

```

bool operator<(const RationalNumber &o) const {
    return num * o.den < o.num * den;          // aritmetica inteira pura
}

```

Erro Comum de Programação

Comparação `a == b` para frações é *simples* se ambas estão na forma reduzida: basta comparar `num == num && den == den`. É uma das vantagens de manter sempre a forma reduzida: tornam-se válidos atalhos sintáticos de igualdade.

6 Exercício 19.11 — Classe Polynomial

Enunciado. Desenvolva a classe `Polynomial`. Representação interna: array de termos, cada um com coeficiente e expoente. Sobrecarregue: (a) `+`; (b) `-`; (c) `=`; (d) `*`; (e) `+=`, `-=` e `*=`.

Análise do problema

O polinômio $3x^4 + 2x^2 - 5$ será armazenado como *três* termos: $(3, 4), (2, 2), (-5, 0)$. Esta representação *esparsa* é mais eficiente que um array de coeficientes com índice = expoente, pois polinômios reais frequentemente têm muitos coeficientes zero.

Decisões de design:

- Os termos são armazenados em ordem decrescente de expoente (forma canônica).
- Termos com coeficiente 0 são removidos automaticamente (não aparecem na representação).
- Cada classe `Polynomial` possui um array *dinâmico* de `Termos` — requer construtor de cópia, destrutor e `operator=`.

Cabeçalho `Polynomial.h`

```

1 // Exercício 19.11 - Polynomial.h
2 // Polinômio armazenado como array dinâmico de termos.
3 // Cada termo tem coeficiente (double) e expoente (int).
4 #ifndef POLYNOMIAL_H
5 #define POLYNOMIAL_H
6
7 #include <iostream>
8
9 struct Termo {
10     double coeficiente;
11     int     expoente;
12 };
13
14 class Polynomial {
15     friend std::ostream &operator<<(std::ostream &out, const Polynomial &p);
16 public:
17     Polynomial(); // construtor default
18     Polynomial(const Termo *termos, int qtd); // por array de termos
19     Polynomial(const Polynomial &outro); // cópia
20     ~Polynomial(); // destrutor
21
22     // Setters / getters
23     int getNumeroDeTermos() const;
24     double getCoeficiente(int i) const;
25     int getExpoente(int i) const;
26     void setTermo(int expoente, double coeficiente);
27
28     // Operadores aritméticos
29     Polynomial operator+(const Polynomial &o) const;
30     Polynomial operator-(const Polynomial &o) const;
31     Polynomial operator*(const Polynomial &o) const;
32
33     // Atribuição e compostos
34     const Polynomial &operator=(const Polynomial &o);
35     const Polynomial &operator+=(const Polynomial &o);
36     const Polynomial &operator-=(const Polynomial &o);
37     const Polynomial &operator*=(const Polynomial &o);
38
39 private:

```

```

40     Termo *termos;           // array dinamico de termos (somente termos != 0)
41     int     qtd;             // quantos termos sao usados
42
43     void normalizar();       // remove termos com coeficiente 0 e ordena
44 };
45
46 #endif

```

Implementação Polynomial.cpp

```

1  // Exercício 19.11 - Polynomial.cpp
2  #include <algorithm>
3  #include <cmath>
4  #include "Polynomial.h"
5  using namespace std;
6
7  Polynomial::Polynomial() : termos(nullptr), qtd(0) {}
8
9  Polynomial::Polynomial(const Termo *t, int n) : qtd(n) {
10     termos = new Termo[qtd];
11     for (int i = 0; i < qtd; ++i) termos[i] = t[i];
12     normalizar();
13 }
14
15 Polynomial::Polynomial(const Polynomial &o) : qtd(o.qtd) {
16     termos = new Termo[qtd];
17     for (int i = 0; i < qtd; ++i) termos[i] = o.termos[i];
18 }
19
20 Polynomial::~Polynomial() {
21     delete[] termos;
22 }
23
24 int     Polynomial::getNumeroDeTermos() const { return qtd; }
25 double Polynomial::getCoeficiente(int i) const { return termos[i].coeficiente; }
26 int     Polynomial::getExpoente(int i)      const { return termos[i].expoente; }
27
28 // Insere/atualiza um termo: se ja existe um termo com o mesmo expoente,
29 // substitui o coeficiente; senao, adiciona
30 void Polynomial::setTermo(int expoente, double coeficiente) {
31     for (int i = 0; i < qtd; ++i) {
32         if (termos[i].expoente == expoente) {
33             termos[i].coeficiente = coeficiente;
34             normalizar();
35             return;
36         }
37     }
38     // Nao existe -> cria espaco maior e adiciona
39     Termo *novo = new Termo[qtd + 1];
40     for (int i = 0; i < qtd; ++i) novo[i] = termos[i];
41     novo[qtd].expoente = expoente;
42     novo[qtd].coeficiente = coeficiente;
43     delete[] termos;
44     termos = novo;
45     ++qtd;
46     normalizar();
47 }
48
49 // Remove termos com coeficiente 0 e ordena por expoente decrescente
50 void Polynomial::normalizar() {
51     // Remove zeros (em um polinomio temporariamente "sujo")
52     int j = 0;

```

```

53     for (int i = 0; i < qtd; ++i)
54         if (termos[i].coeficiente != 0.0)
55             termos[j++] = termos[i];
56     qtd = j;
57     // Ordena por expoente decrescente
58     sort(termos, termos + qtd, [](const Termo &a, const Termo &b) {
59         return a.expoente > b.expoente;
60     });
61 }
62
63 Polynomial Polynomial::operator+(const Polynomial &o) const {
64     Polynomial r;
65     // Reserva tamanho maximo
66     r.termos = new Termo[qtd + o.qtd];
67     r.qtd = 0;
68     // Copia todos os termos de *this
69     for (int i = 0; i < qtd; ++i) r.termos[r.qtd++] = termos[i];
70     // Adiciona ou soma de o
71     for (int i = 0; i < o.qtd; ++i) {
72         bool achou = false;
73         for (int k = 0; k < r.qtd; ++k) {
74             if (r.termos[k].expoente == o.termos[i].expoente) {
75                 r.termos[k].coeficiente += o.termos[i].coeficiente;
76                 achou = true;
77                 break;
78             }
79         }
80         if (!achou) r.termos[r.qtd++] = o.termos[i];
81     }
82     r.normalizar();
83     return r;
84 }
85
86 Polynomial Polynomial::operator-(const Polynomial &o) const {
87     Polynomial neg(o); // copia o
88     for (int i = 0; i < neg.qtd; ++i) neg.termos[i].coeficiente *= -1;
89     return *this + neg;
90 }
91
92 Polynomial Polynomial::operator*(const Polynomial &o) const {
93     Polynomial r;
94     r.termos = new Termo[qtd * o.qtd];
95     r.qtd = 0;
96     for (int i = 0; i < qtd; ++i) {
97         for (int j = 0; j < o.qtd; ++j) {
98             int expProd = termos[i].expoente + o.termos[j].expoente;
99             double cProd = termos[i].coeficiente * o.termos[j].coeficiente;
100             // Verifica se ja temos esse expoente
101             bool achou = false;
102             for (int k = 0; k < r.qtd; ++k) {
103                 if (r.termos[k].expoente == expProd) {
104                     r.termos[k].coeficiente += cProd;
105                     achou = true; break;
106                 }
107             }
108             if (!achou) {
109                 r.termos[r.qtd].expoente = expProd;
110                 r.termos[r.qtd].coeficiente = cProd;
111                 ++r.qtd;
112             }
113         }
114     }
115     r.normalizar();
116     return r;

```



```

117 }
118
119 const Polynomial &Polynomial::operator=(const Polynomial &o) {
120     if (this != &o) {
121         delete[] termos;
122         qtd = o.qtd;
123         termos = new Termo[qtd];
124         for (int i = 0; i < qtd; ++i) termos[i] = o.termos[i];
125     }
126     return *this;
127 }
128
129 const Polynomial &Polynomial::operator+=(const Polynomial &o) {
130     *this = *this + o;
131     return *this;
132 }
133
134 const Polynomial &Polynomial::operator-=(const Polynomial &o) {
135     *this = *this - o;
136     return *this;
137 }
138
139 const Polynomial &Polynomial::operator*=(const Polynomial &o) {
140     *this = *this * o;
141     return *this;
142 }
143
144 // Impressao em formato canonico: 3x^2 + 2x - 5
145 ostream &operator<<(ostream &out, const Polynomial &p) {
146     if (p.qtd == 0) { out << "0"; return out; }
147     for (int i = 0; i < p.qtd; ++i) {
148         double c = p.termos[i].coeficiente;
149         int e = p.termos[i].expoente;
150
151         // Sinal
152         if (i == 0) {
153             if (c < 0) out << "-";
154         } else {
155             out << (c >= 0 ? " + " : " - ");
156         }
157         double cAbs = std::fabs(c);
158
159         // Coeficiente (omite se for 1 e o expoente nao for 0)
160         if (cAbs != 1.0 || e == 0) out << cAbs;
161
162         // Variavel e expoente
163         if (e == 0) {
164             ;
165         } else if (e == 1) {
166             out << "x";
167         } else {
168             out << "x^" << e;
169         }
170     }
171     return out;
172 }

```

Programa de teste

```

1 // Exercicio 19.11 - main_Polynomial.cpp
2 #include <iostream>
3 #include "Polynomial.h"

```

```

4 using namespace std;
5
6 int main() {
7     // p1 = 3x^2 + 2x + 1
8     Termo t1[] = {{3, 2}, {2, 1}, {1, 0}};
9     Polynomial p1(t1, 3);
10    cout << "p1 = " << p1 << endl;
11
12    // p2 = x^3 - 4x + 7
13    Termo t2[] = {{1, 3}, {-4, 1}, {7, 0}};
14    Polynomial p2(t2, 3);
15    cout << "p2 = " << p2 << endl;
16
17    // p3 = 5x^2 - 2x + 3
18    Termo t3[] = {{5, 2}, {-2, 1}, {3, 0}};
19    Polynomial p3(t3, 3);
20    cout << "p3 = " << p3 << endl;
21
22    cout << "\n=== Soma ===" << endl;
23    Polynomial soma = p1 + p2;
24    cout << "p1 + p2 = " << soma << endl;
25    // Esperado: x^3 + 3x^2 - 2x + 8
26
27    Polynomial soma13 = p1 + p3;
28    cout << "p1 + p3 = " << soma13 << " (esperado: 8x^2 + 4)" << endl;
29
30    cout << "\n=== Subtracao ===" << endl;
31    Polynomial diff = p1 - p3;
32    cout << "p1 - p3 = " << diff << " (esperado: -2x^2 + 4x - 2)" << endl;
33
34    cout << "\n=== Multiplicacao ===" << endl;
35    // (3x^2+2x+1) * (5x^2-2x+3) =
36    // 15x^4 - 6x^3 + 9x^2 + 10x^3 - 4x^2 + 6x + 5x^2 - 2x + 3 =
37    // 15x^4 + 4x^3 + 10x^2 + 4x + 3
38    Polynomial prod = p1 * p3;
39    cout << "p1 * p3 = " << prod << endl;
40    cout << " (esperado: 15x^4 + 4x^3 + 10x^2 + 4x + 3)" << endl;
41
42    cout << "\n=== Atribuicao composta ===" << endl;
43    Polynomial q = p1;
44    cout << "q = p1 = " << q << endl;
45
46    q += p3;
47    cout << "q += p3 => q = " << q << " (= p1+p3)" << endl;
48
49    q -= p3;
50    cout << "q -= p3 => q = " << q << " (= p1 novamente)" << endl;
51
52    q *= p3;
53    cout << "q *= p3 => q = " << q << " (= p1*p3)" << endl;
54
55    cout << "\n=== Atribuicao = ===" << endl;
56    Polynomial r;
57    r = p2;
58    cout << "r (apos r = p2) = " << r << endl;
59
60    cout << "\n=== setTermo ===" << endl;
61    Polynomial s;
62    s.setTermo(3, 4);
63    s.setTermo(1, -1);
64    s.setTermo(0, 6);
65    cout << "s (4x^3 - x + 6) = " << s << endl;
66    s.setTermo(1, 0); // remove o termo linear
67    cout << "s apos setTermo(1, 0): " << s << " (deve perder o x)" << endl;

```

```

68
69     return 0;
70 }

```

Execução

Saída da execução

```

p1 = 3x^2 + 2x + 1
p2 = x^3 - 4x + 7
p3 = 5x^2 - 2x + 3

=== Soma ===
p1 + p2 = x^3 + 3x^2 - 2x + 8
p1 + p3 = 8x^2 + 4      (esperado: 8x^2 + 4)

=== Subtracao ===
p1 - p3 = -2x^2 + 4x - 2    (esperado: -2x^2 + 4x - 2)

=== Multiplicacao ===
p1 * p3 = 15x^4 + 4x^3 + 10x^2 + 4x + 3
      (esperado: 15x^4 + 4x^3 + 10x^2 + 4x + 3)

=== Atribuicao composta ===
q = p1 = 3x^2 + 2x + 1
q += p3 => q = 8x^2 + 4      (= p1+p3)
q -= p3 => q = 3x^2 + 2x + 1  (= p1 novamente)
q *= p3 => q = 15x^4 + 4x^3 + 10x^2 + 4x + 3  (= p1*p3)

=== Atribuicao = ===
r (apos r = p2) = x^3 - 4x + 7

=== setTermo ===
s (4x^3 - x + 6) = 4x^3 - x + 6
s apos setTermo(1, 0): 4x^3 + 6  (deve perder o x)

```

Conferência matemática

Multiplicação $p_1 \cdot p_3$. Expansão manual:

$$\begin{aligned}
 (3x^2 + 2x + 1)(5x^2 - 2x + 3) &= 15x^4 - 6x^3 + 9x^2 \\
 &\quad + 10x^3 - 4x^2 + 6x \\
 &\quad + 5x^2 - 2x + 3 \\
 &= 15x^4 + 4x^3 + 10x^2 + 4x + 3. \quad \checkmark
 \end{aligned}$$

Discussão — as compostas e a *Regra dos Três*

Implementação das compostas em termos das simples

```

const Polynomial &Polynomial::operator+=(const Polynomial &o) {
    *this = *this + o;          // delega a operator+ e operator=
    return *this;
}

```

É o idioma canônico: $a += b$ significa “ $a = a + b$ e retorne uma referência a a ”. Implementar dessa maneira garante que qualquer alteração no comportamento de $+$ se propague automaticamente para $+=$.

A Regra dos Três

A classe `Polynomial` tem um `Termo` `*termos` *dinâmico*. Por isso é obrigatório implementar manualmente:

1. **Construtor de cópia:** `Polynomial(const Polynomial &)` — aloca um novo array e copia os termos.
2. **Destruitor:** `~Polynomial()` — libera o array com `delete[]`.
3. **Operador de atribuição:** `operator=` — libera o array antigo, aloca novo e copia.

Esquecer qualquer um dos três seria desastroso. Se omitíssemos o construtor de cópia, o compilador geraria um padrão que copia o ponteiro `termos` membro-a-membro — duas instâncias compartilhariam o mesmo array, e a primeira a ser destruída invalidaria a segunda (*double delete*).

Observação de Engenharia de Software

A *Regra dos Três* em C++03 tornou-se a *Regra dos Cinco* em C++11 com a introdução de *move semantics*: `Polynomial(Polynomial &&)` (construtor de movimento) e `operator=(Polynomial &&)` (atribuição de movimento). Esses dois transferem propriedade de recursos em vez de copiar — evitando alocações redundantes em situações como `p = p1 + p2`. C++ moderno gerencia isso automaticamente em muitos casos, mas em classes com recursos manuais (como `Polynomial`), é melhor implementar as cinco funções explicitamente.

Boa Prática de Programação

A formatação de um polinômio para impressão tem muitos casos especiais:

- Coeficiente 1 omite-se: $1x^2$ vira x^2 .
- Coeficiente -1 vira “-”: $-1x$ vira $-x$.
- Expoente 0 omite-se: $4x^0$ vira 4.
- Expoente 1 omite-se: $4x^1$ vira $4x$.
- Sinal entre termos: “+” ou “-” precedido de espaço.

Nossa implementação trata todos esses casos. Vale destacar quanto trabalho está por trás de uma saída “elegante”.

Encerramento

Os seis exercícios deste capítulo nos fizeram exercitar todos os principais idiomas de sobrecarga em C++:

- Operadores aritméticos retornando objetos novos por valor (`Complex`, `Rational`, `HugeInt`, `Polynomial`).
- Operadores compostos (`+=`, `-=`, `*=`) retornando referências para encadeamento.
- Operadores relacionais retornando `bool`.
- Operadores de I/O (`<<` e `>>`) como funções amigas, retornando `ostream&/istream&` para encadeamento.
- Operador de chamada de função (`operator()`) em duas versões para suportar leitura e escrita.
- Gerenciamento manual de memória dinâmica com a *Regra dos Três*.

Com isso encerramos o tratamento de classes em sua forma canônica. O Capítulo 20 introduzirá **herança**, permitindo construir uma classe a partir de outra com mínima duplicação — e finalmente realizar o clássico relacionamento *é-um* (em contraste com o *tem-um* que vimos no Cap. 18 através de composição).
